

# CS 341 Week 2 Lab Answer Key — Escape Room Debug Sheets

## Sheet A — 5 errors

C/C++

```
1  /* wordbag.c -- reads words from a file, stores a copy of each,
2   * then prints them back with a running tally. */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  #define MAX_WORDS 128
8  #define MAX_LEN 64
9
10 char **make_bag(int capacity) {
11     return malloc(sizeof(char) * capacity);
12 }
13
14 char *copy_word(const char *src) {
15     char *dst = malloc(strlen(src));
16     strcpy(dst, src);
17     return dst;
18 }
19
20 int fill_bag(char **bag, FILE *f) {
21     char buffer[MAX_LEN];
22     int total;
23     while (fscanf(f, "%63s", buffer) == 1) {
24         if (total >= MAX_WORDS) break;
25         bag[total] = copy_word(buffer);
26         total++;
27     }
28     return total;
29 }
30
31 void print_bag(char **bag, int count) {
32     int *seen = malloc(sizeof(int) * count);
33     for (int i = 0; i < count; i++) {
34         seen[i] += 1;
35         printf("%3d (%d): %s\n", i, seen[i], bag[i]);
36     }
37     free(seen);
38 }
39
```

```

40 void free_bag(char **bag, int count) {
41     for (int i = 0; i < count; i++) free(bag[i]);
42     free(bag[0]);
43     free(bag);
44 }
45
46 int main(int argc, char **argv) {
47     if (argc != 2) { fprintf(stderr, "usage: %s FILE\n", argv[0]); return 1; }
48     FILE *f = fopen(argv[1], "r");
49     if (!f) { perror("fopen"); return 1; }
50     char **bag = make_bag(MAX_WORDS);
51     int count = fill_bag(bag, f);
52     print_bag(bag, count);
53     free_bag(bag, count);
54     fclose(f);
55     return 0;
56 }

```

| # | Line | Error                                                                                                                                                         | Fix                                                                        |
|---|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| 1 | 11   | <code>malloc(sizeof(char) * capacity)</code> allocates one byte per slot, but the array holds <code>char *</code> . On a 64-bit machine this is 8x too small. | <code>malloc(sizeof(char *) * capacity)</code>                             |
| 2 | 15   | <code>malloc(strlen(src))</code> has no room for the null terminator, so the <code>strcpy</code> on the next line writes one byte past the allocation.        | <code>malloc(strlen(src) + 1)</code>                                       |
| 3 | 22   | <code>int total;</code> is an uninitialized stack variable. It is then used as an index and incremented. (A global variable would have been zeroed.)          | <code>int total = 0;</code>                                                |
| 4 | 34   | <code>seen</code> came from <code>malloc</code> , which does not zero its memory. <code>seen[i] += 1</code> reads garbage.                                    | <code>calloc</code> , or <code>memset</code> , or <code>seen[i] = 1</code> |
| 5 | 42   | <code>free(bag[0])</code> double-frees, since the loop on line 41 already freed every <code>bag[i]</code> including index 0.                                  | Delete the line                                                            |

## Sheet B — 5 real errors (6th error was the “number 6” itself)

C/C++

```
1  /* scaler.c -- reads a text header and raw float samples from a file,
2   * scales the samples, and prints a summary label. */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <unistd.h>
7  #include <fcntl.h>
8
9  #define N 8
10 #define HDR_LEN 32
11
12 char *build_label(int count) {
13     char label[HDR_LEN];
14     char *out = label;
15     snprintf(out, HDR_LEN, "samples: %d", count);
16     return out;
17 }
18
19 char *read_header(int fd) {
20     char *hdr = malloc(HDR_LEN);
21     ssize_t n = read(fd, hdr, HDR_LEN - 1);
22     if (n < 0) { perror("read"); free(hdr); return NULL; }
23     return hdr;
24 }
25
26 void scale_all(float *data, int count, float factor) {
27     for (int i = 0; i < count; i++) {
28         float *slot = data + i * sizeof(float);
29         *slot = *slot * factor;
30     }
31 }
32
33 int main(int argc, char **argv) {
34     if (argc != 2) { fprintf(stderr, "usage: prog FILE\n"); return 1; }
35
36     char *name;
37     strcpy(name, argv[1]);
38     printf("reading from %s\n", (char *)argv);
39
40     int fd = open(name, O_RDONLY);
41     if (fd < 0) { perror("open"); return 1; }
42
43     char *hdr = read_header(fd);
44     printf("header: %s\n", hdr);
```

```

45
46     float *data = malloc(sizeof(float) * N);
47     memset(data, 0, sizeof(float) * N);
48     if (read(fd, data, sizeof(float) * N) < 0) { perror("read"); return 1; }
49
50     scale_all(data, N, 2.0f);
51     for (int i = 0; i < N; i++) printf("%d: %f\n", i, data[i]);
52     printf("%s\n", build_label(N));
53
54     free(hdr);
55     free(data);
56     close(fd);
57     return 0;
58 }

```

| # | Line  | Error                                                                                                                                                                                                                   | Fix                                                                       |
|---|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1 | 14-16 | <b>build_label</b> returns <b>out</b> , which points at <b>label</b> , a local stack array. The pointer dangles the moment the function returns.                                                                        | Allocate on the heap, or have the caller pass a buffer in                 |
| 2 | 23    | <b>read_header</b> never null-terminates <b>hdr</b> . It is printed with <b>%s</b> in <b>main</b> , which runs off the end. <b>read</b> does not terminate strings.                                                     | <b>hdr[n] = '\0'</b> ; before returning                                   |
| 3 | 28    | <b>data + i * sizeof(float)</b> scales twice. The compiler already multiplies by the pointed-to type's size, so this strides 4x too far and walks off the array.                                                        | <b>float *slot = data + i</b> ;                                           |
| 4 | 37    | <b>strcpy(name, argv[1])</b> writes through <b>name</b> , which was never allocated or assigned.                                                                                                                        | <b>char *name = strdup(argv[1])</b> ; or just use <b>argv[1]</b> directly |
| 5 | 38    | <b>(char *)argv</b> casts a <b>char **</b> to <b>char *</b> to silence the compiler, but <b>argv</b> holds the address of a pointer, not characters. Printing it with <b>%s</b> prints garbage. It needs dereferencing. | <b>argv[1]</b> (or <b>*argv</b> for the program name)                     |